

ML-1 Mini Project: Data Pre-processing and EDA

Topic: Autism Prediction

Name: Krisha Shah

Sap Id: 60009220055

Roll No: D065

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
training_data = pd.read_csv('/content/train.csv')
```

PRE-PROCESSING

```
training_data.head(10)
```

	ID	A1_Score	A2_Score	A3_Score	A4_Score	A5_Score	A6_Score	A7_Score	A8_Score	A9_Score
0	1	1	0	1	0	1	0	1	0	
1	2	0	0	0	0	0	0	0	0	
2	3	1	1	1	1	1	1	1	1	
3	4	0	0	0	0	0	0	0	0	
4	5	0	0	0	0	0	0	0	0	
5	6	1	0	0	0	0	1	0	0	
6	7	1	0	0	0	0	0	1	1	
7	8	1	1	1	1	1	1	1	0	
8	9	1	1	1	1	0	0	0	1	
9	10	0	0	0	0	0	0	0	1	

10 rows x 22 columns

```
training_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 22 columns):
#   Column              Non-Null Count  Dtype
---  -
0   ID                   800 non-null   int64
1   A1_Score             800 non-null   int64
2   A2_Score             800 non-null   int64
3   A3_Score             800 non-null   int64
4   A4_Score             800 non-null   int64
5   A5_Score             800 non-null   int64
6   A6_Score             800 non-null   int64
7   A7_Score             800 non-null   int64
8   A8_Score             800 non-null   int64
9   A9_Score             800 non-null   int64
10  A10_Score            800 non-null   int64
11  age                  800 non-null   float64
12  gender               800 non-null   object
13  ethnicity            800 non-null   object
14  jaundice             800 non-null   object
```

```

15  austim          800 non-null  object
16  contry_of_res   800 non-null  object
17  used_app_before 800 non-null  object
18  result          800 non-null  float64
19  age_desc        800 non-null  object
20  relation        800 non-null  object
21  Class/ASD       800 non-null  int64
dtypes: float64(2), int64(12), object(8)
memory usage: 137.6+ KB

```

```
training_data.describe()
```

	ID	A1_Score	A2_Score	A3_Score	A4_Score	A5_Score	A6_Score	A
count	800.0000	800.000000	800.000000	800.000000	800.000000	800.000000	800.000000	800
mean	400.5000	0.560000	0.530000	0.450000	0.41500	0.395000	0.303750	0
std	231.0844	0.496697	0.499411	0.497805	0.49303	0.489157	0.460164	0
min	1.0000	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0
25%	200.7500	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0
50%	400.5000	1.000000	1.000000	0.000000	0.00000	0.000000	0.000000	0
75%	600.2500	1.000000	1.000000	1.000000	1.00000	1.000000	1.000000	1
max	800.0000	1.000000	1.000000	1.000000	1.00000	1.000000	1.000000	1

```
training_data.shape
```

```
(800, 22)
```

```
training_data.isnull().sum()
```

```

ID          0
A1_Score    0
A2_Score    0
A3_Score    0
A4_Score    0
A5_Score    0
A6_Score    0
A7_Score    0
A8_Score    0
A9_Score    0
A10_Score   0
age         0
gender      0
ethnicity   0
jaundice    0
austim      0
contry_of_res 0
used_app_before 0
result      0
age_desc    0
relation    0
Class/ASD   0
dtype: int64

```

No null values in the above dataset

```

duplicate_rows = training_data[training_data.duplicated()]
if duplicate_rows.empty:
    print("No duplicate rows found.")
else:
    print("Duplicate Rows:")
    print(duplicate_rows)

    No duplicate rows found.

```

```

categorical_columns = training_data.select_dtypes(include=['object']).columns
inconsistent_values = {}
for col in categorical_columns:
    unique_values = training_data[col].unique()
    if len(unique_values) > 1:
        inconsistent_values[col] = unique_values
if inconsistent_values:
    print("Inconsistent Values:")
    for col, values in inconsistent_values.items():
        print(f"{col}: {values}")
else:
    print("No inconsistent values found.")

Inconsistent Values:
gender: ['f' 'm']
ethnicity: ['?' 'White-European' 'Middle Eastern ' 'Pasifika' 'Black' 'Others'
'Hispanic' 'Asian' 'Turkish' 'South Asian' 'Latino' 'others']
jaundice: ['no' 'yes']
austim: ['no' 'yes']
contry_of_res: ['Austria' 'India' 'United States' 'South Africa' 'Jordan'
'United Kingdom' 'Brazil' 'New Zealand' 'Canada' 'Kazakhstan'
'United Arab Emirates' 'Australia' 'Ukraine' 'Iraq' 'France' 'Malaysia'
'Viet Nam' 'Egypt' 'Netherlands' 'Afghanistan' 'Oman' 'Italy'
'AmericanSamoa' 'Bahamas' 'Saudi Arabia' 'Ireland' 'Aruba' 'Sri Lanka'
'Russia' 'Bolivia' 'Azerbaijan' 'Armenia' 'Serbia' 'Ethiopia' 'Sweden'
'Iceland' 'Hong Kong' 'Angola' 'China' 'Germany' 'Spain' 'Tonga'
'Pakistan' 'Iran' 'Argentina' 'Japan' 'Mexico' 'Nicaragua' 'Sierra Leone'
'Czech Republic' 'Niger' 'Romania' 'Cyprus' 'Belgium' 'Burundi'
'Bangladesh']
used_app_before: ['no' 'yes']
relation: ['Self' 'Relative' 'Parent' '?' 'Others' 'Health care professional']

training_data['gender'] = training_data['gender'].map({'f': 'Female', 'm': 'Male'})
training_data['ethnicity'] = training_data['ethnicity'].replace('others', 'Others').str.strip() # Remove trailing whitespaces
training_data['relation'] = training_data['relation'].replace('?', 'Unknown')

training_data.columns

Index(['ID', 'A1_Score', 'A2_Score', 'A3_Score', 'A4_Score', 'A5_Score',
'A6_Score', 'A7_Score', 'A8_Score', 'A9_Score', 'A10_Score', 'age',
'gender', 'ethnicity', 'jaundice', 'austim', 'contry_of_res',
'used_app_before', 'result', 'age_desc', 'relation', 'Class/ASD'],
dtype='object')

# Replace '?' with NaN
training_data.replace('?', np.nan, inplace=True)

# Drop rows with NaN values
training_data.dropna(inplace=True)

# Display the modified DataFrame
print(training_data)

```

	ID	A1_Score	A2_Score	A3_Score	A4_Score	A5_Score	A6_Score	\
2	3	1	1	1	1	1	1	
5	6	1	0	0	0	0	1	
6	7	1	0	0	0	0	0	
7	8	1	1	1	1	1	1	
8	9	1	1	1	1	0	0	
..	...	...	...	...	...	...	...	
790	791	1	0	1	1	1	1	
791	792	0	0	0	1	1	1	
795	796	0	1	0	0	0	0	
796	797	0	1	1	0	0	1	
797	798	0	0	0	0	0	0	

	A7_Score	A8_Score	A9_Score	...	gender	ethnicity	jaundice	\
2	1	1	1	...	Male	White-European	no	
5	0	0	1	...	Male	Middle Eastern	no	
6	1	1	1	...	Female	Pasifika	no	
7	1	0	1	...	Male	Black	no	
8	0	1	1	...	Male	White-European	no	
..	...	...	...	...	...	...	...	
790	1	0	1	...	Male	Others	no	
791	0	0	0	...	Male	Black	no	
795	0	0	1	...	Male	Hispanic	no	
796	0	1	1	...	Male	White-European	no	
797	0	0	0	...	Male	South Asian	yes	

	austim	contry_of_res	used_app_before	result	age_desc	relation \
2	yes	United States	no	14.851484	18 and more	Self
5	no	Jordan	no	9.562117	18 and more	Self
6	no	United Kingdom	no	7.984569	18 and more	Self
7	yes	United States	no	13.237898	18 and more	Self
8	no	Brazil	no	-1.755774	18 and more	Self
..	...	...	...	...	...	...
790	no	India	no	4.846236	18 and more	Self
791	no	Australia	no	8.687281	18 and more	Self
795	no	New Zealand	no	12.999501	18 and more	Self
796	no	Cyprus	no	13.561518	18 and more	Self
797	no	New Zealand	no	2.653177	18 and more	Self

	Class/ASD
2	1
5	0
6	0
7	1
8	0
..	...
790	0
791	0
795	0
796	0
797	0

[597 rows x 22 columns]

Therefore the dataset is clean and can be worked upon.

Label Encoding to convert 'gender' values to numeric to use them in the EDA section

```
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()
training_data['gender_encoded'] = label_encoder.fit_transform(training_data['gender'])

print(training_data[['gender', 'gender_encoded']])
```

	gender	gender_encoded
2	Male	1
5	Male	1
6	Female	0
7	Male	1
8	Male	1
..	...	...
790	Male	1
791	Male	1
795	Male	1
796	Male	1
797	Male	1

[597 rows x 2 columns]

Hence we see that after encoding, all the 'Male' values have been encoded to 1 and 'Female' values to 0.

Label Encoding to convert country names to numeric data type.

```
label_encoder_country = LabelEncoder()
training_data['country_encoded'] = label_encoder_country.fit_transform(training_data['contry_of_res'])

# Display the modified DataFrame
print(training_data[['contry_of_res', 'country_encoded']])
```

	contry_of_res	country_encoded
2	United States	53
5	Jordan	30
6	United Kingdom	52
7	United States	53
8	Brazil	12
..	...	...
790	India	24
791	Australia	5

795	New Zealand	35
796	Cyprus	16
797	New Zealand	35

[597 rows x 2 columns]

```
label_encoder_ethnicity = LabelEncoder()
training_data['ethnicity_encoded'] = label_encoder_country.fit_transform(training_data['ethnicity'])
```

```
# Display the modified DataFrame
print(training_data[['ethnicity', 'ethnicity_encoded']])
```

	ethnicity	ethnicity_encoded
2	White-European	9
5	Middle Eastern	4
6	Pasifika	6
7	Black	1
8	White-European	9
..	...	...
790	Others	5
791	Black	1
795	Hispanic	2
796	White-European	9
797	South Asian	7

[597 rows x 2 columns]

```
label_encoder_jaundice = LabelEncoder()
training_data['jaundice_encoded'] = label_encoder.fit_transform(training_data['jaundice'])
```

```
print(training_data[['jaundice', 'jaundice_encoded']])
```

```
label_encoder_autism = LabelEncoder()
training_data['austim_encoded'] = label_encoder.fit_transform(training_data['austim'])
```

```
print(training_data[['austim', 'austim_encoded']])
```

	jaundice	jaundice_encoded
2	no	0
5	no	0
6	no	0
7	no	0
8	no	0
..	...	...
790	no	0
791	no	0
795	no	0
796	no	0
797	yes	1

[597 rows x 2 columns]

	austim	austim_encoded
2	yes	1
5	no	0
6	no	0
7	yes	1
8	no	0
..	...	...
790	no	0
791	no	0
795	no	0
796	no	0
797	no	0

[597 rows x 2 columns]

```
output = []
for col in training_data.columns:
    unique = training_data[col].nunique()
    colType = str(training_data[col].dtype)
    categories=training_data[col].unique()

    output.append([col, unique, colType, categories])

output = pd.DataFrame(output)
output.columns = ['colName', 'unique', 'dtype', 'categories']
output
```

	colName	unique	dtype	categories	
0	ID	597	int64	[3, 6, 7, 8, 9, 10, 14, 15, 17, 18, 20, 21, 22...	
1	A1_Score	2	int64	[1, 0]	
2	A2_Score	2	int64	[1, 0]	
3	A3_Score	2	int64	[1, 0]	
4	A4_Score	2	int64	[1, 0]	
5	A5_Score	2	int64	[1, 0]	
6	A6_Score	2	int64	[1, 0]	
7	A7_Score	2	int64	[1, 0]	
8	A8_Score	2	int64	[1, 0]	
9	A9_Score	2	int64	[1, 0]	
10	A10_Score	2	int64	[1, 0]	
11	age	597	float64	[7.380373076, 31.5279644, 28.42797076, 26.4844...	
12	gender	2	object	[Male, Female]	
13	ethnicity	10	object	[White-European, Middle Eastern, Pasifika, Bla...	
14	jaundice	2	object	[no, yes]	
15	austim	2	object	[yes, no]	
16	contry_of_res	55	object	[United States, Jordan, United Kingdom, Brazil...	
17	used_app_before	2	object	[no, yes]	
18	result	597	float64	[14.85148447, 9.562117428, 7.984569116, 13.237...	
19	age_desc	1	object	[18 and more]	
20	relation	6	object	[Self, Parent, Unknown, Relative, Others, Heal...	
21	Class/ASD	2	int64	[1, 0]	
22	gender_encoded	2	int64	[1, 0]	
23	country_encoded	55	int64	[53, 30, 52, 12, 35, 51, 24, 31, 26, 20, 32, 5...	
24	ethnicity_encoded	10	int64	[9, 4, 6, 1, 5, 2, 0, 8, 7, 3]	
25	jaundice_encoded	2	int64	[0, 1]	
26	austim_encoded	2	int64	[1, 0]	

Next steps:

[Generate code with output](#)

☒ [View recommended plots](#)

▼ EDA

```

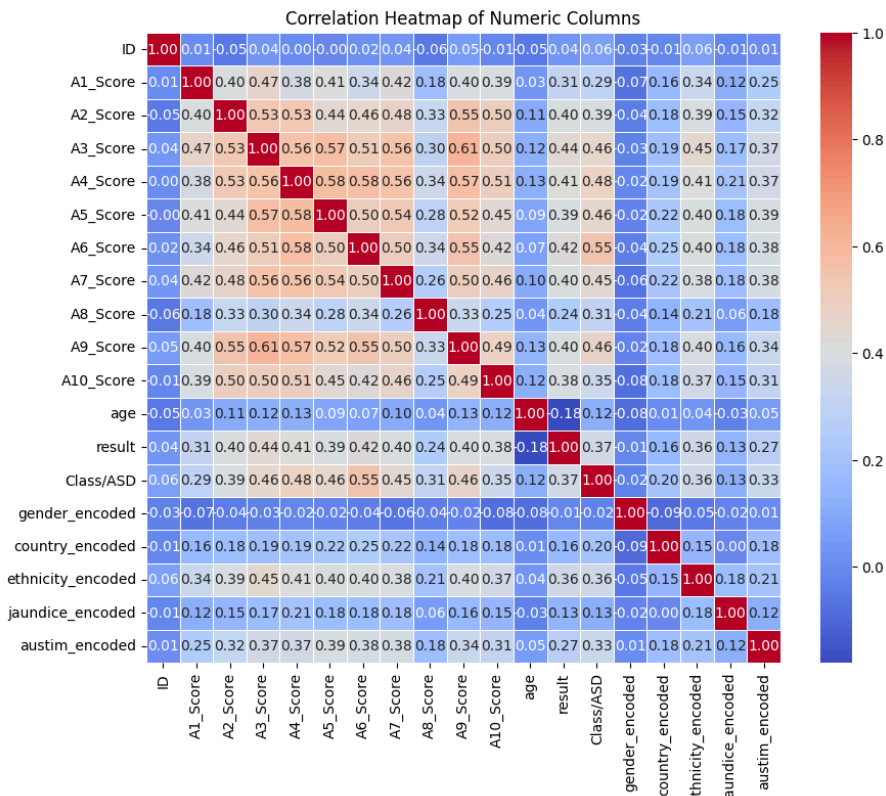
# Select only numeric columns
numeric_columns = training_data.select_dtypes(include=['int64', 'float64']).columns

# Create a subset of DataFrame with numeric columns
numeric_data = training_data[numeric_columns]

# Calculate correlation matrix
correlation_matrix = numeric_data.corr()

# Plot heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Correlation Heatmap of Numeric Columns')
plt.show()

```



ANALYSIS:

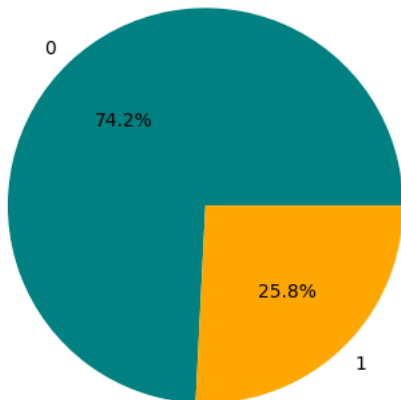
The following inferences can be drawn from the above heatmap:

1. There is a negative correlation between the columns 'age' and 'result', which suggests that as age increases, the possibility of a person being diagnosed with autism decreases. This means that this disease is detected at a very early age, possibly by 2 years.
2. There is a slightly negative correlation between 'gender_encoded' and 'A2_score', which suggests that the attribute score of a particular condition (A2_score) is found specifically in only one gender, which can be studied further in the analysis.
3. Again, there is a slightly negative correlation between 'gender' and 'country', which suggests that in some particular countries, specific genders are found to be diagnosed with autism.
4. A positive correlation can be seen between all the attribute scores 'A1, A2, A3..., A10' and 'age', which suggests that these attribute scores are dependent on the ages of the children.

5. Finally, 'Class/ASD' almost has a positive correlation with all the factors, which suggests that all the factors in the dataset are somewhat equally important in determining autism.

```
colors = ['#008080', '#FFA700', '#FF7F50']
# Calculate the value counts for 'Class/ASD'
class_counts = training_data['Class/ASD'].value_counts()

# Plot the pie chart
plt.pie(class_counts.values, labels=class_counts.index, autopct='%1.1f%%', colors=colors)
plt.show()
```

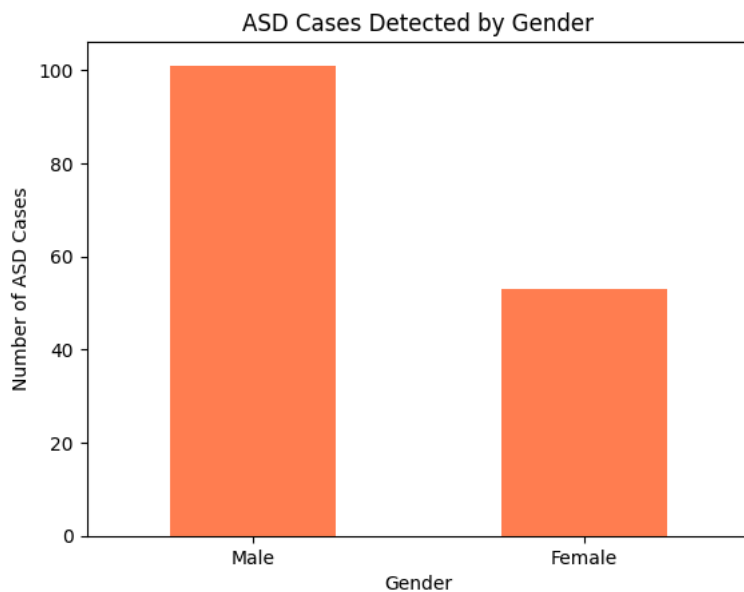


✓ ANALYSIS:

The above piechart simply depicts the distribution of 1s and 0s, that is there are clearly more number of cases where no autism was detected and fewer cases where it was detected.

```
# Count the number of ASD cases for each gender
gender_asd_counts = training_data[training_data['Class/ASD'] == 1]['gender'].value_counts()

# Plot the bar chart
gender_asd_counts.plot(kind='bar', color='coral')
plt.xlabel('Gender')
plt.ylabel('Number of ASD Cases')
plt.title('ASD Cases Detected by Gender')
plt.xticks(rotation=0) # Rotate x-axis labels if needed
plt.show()
```



✓ ANALYSIS:

We can clearly see from the above bar graph that there have been more cases where ASD was detected in males than in females.

```
import matplotlib.pyplot as plt

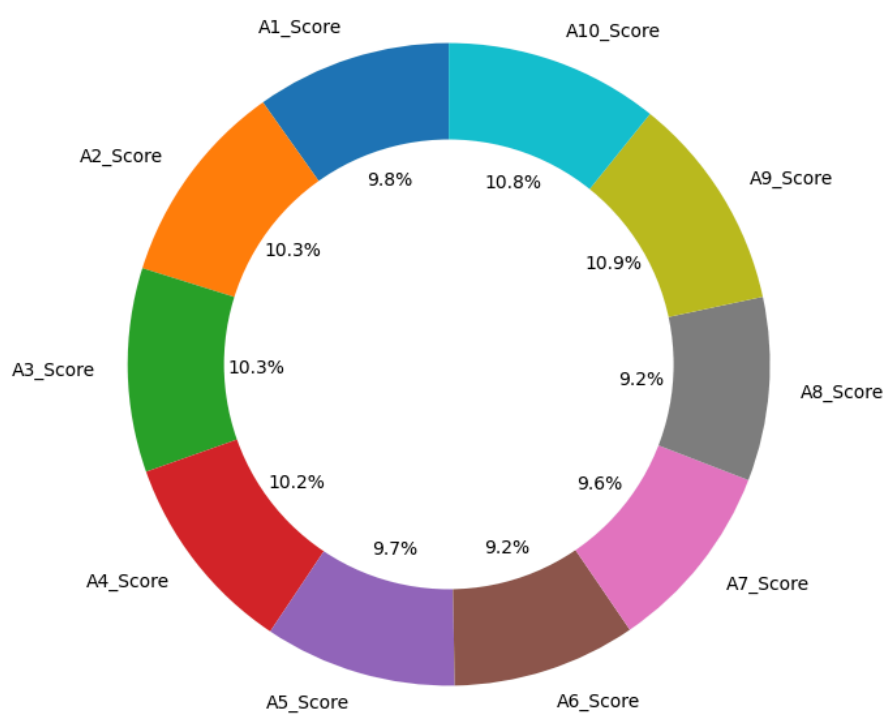
# Select ASD cases from the training data
asd_cases = training_data[training_data['Class/ASD'] == 1]

# Select only the columns containing A1 to A10 scores
a_scores = asd_cases[['A1_Score', 'A2_Score', 'A3_Score', 'A4_Score', 'A5_Score',
                      'A6_Score', 'A7_Score', 'A8_Score', 'A9_Score', 'A10_Score']]

# Calculate the sum of scores for each attribute
attribute_totals = a_scores.sum()

# Plot the doughnut chart
plt.figure(figsize=(8, 8))
plt.pie(attribute_totals, labels=attribute_totals.index, autopct='%1.1f%%', startangle=90, colors=plt.cm.tab10.colors)
centre_circle = plt.Circle((0,0),0.70,fc='white')
fig = plt.gcf()
fig.gca().add_artist(centre_circle)
plt.title('Distribution of A1 to A10 Scores for ASD Cases')
plt.show()
```

Distribution of A1 to A10 Scores for ASD Cases



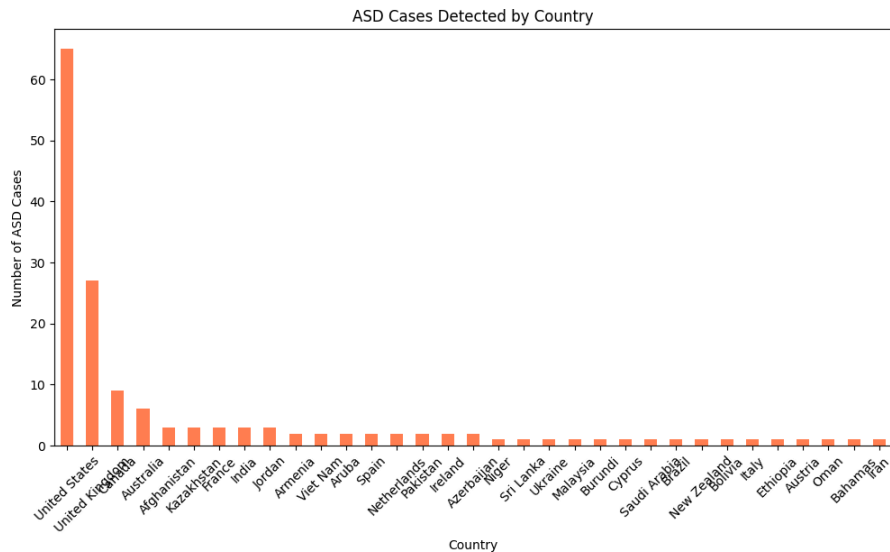
✓ ANALYSIS:

From the above doughnut graph, we can decipher that not all the attribute scores have same kind of impact on the ASD detection. As per the relation, we can see that A9 has the maximum correlation, followed by A10, A2, A3 and so on and A6 and A7 have the least correlation amongst all.

```
import matplotlib.pyplot as plt

# Count the number of ASD cases for each country
country_asd_counts = training_data[training_data['Class/ASD'] == 1]['contry_of_res'].value_counts()

# Plot the bar chart
plt.figure(figsize=(12, 6))
country_asd_counts.plot(kind='bar', color='coral')
plt.xlabel('Country')
plt.ylabel('Number of ASD Cases')
plt.title('ASD Cases Detected by Country')
plt.xticks(rotation=45)
plt.show()
```



The above bar graph represents the distribution of ASD Cases country-wise. it can be hence deciphered that the United states have the maximum number of cases, followed by the United Kingdom, Canada, Australia and Afghanistan. Whereas the least amount of cases have been recorded in Iran, Bahamas, Oman, Austria and Ethiopia.

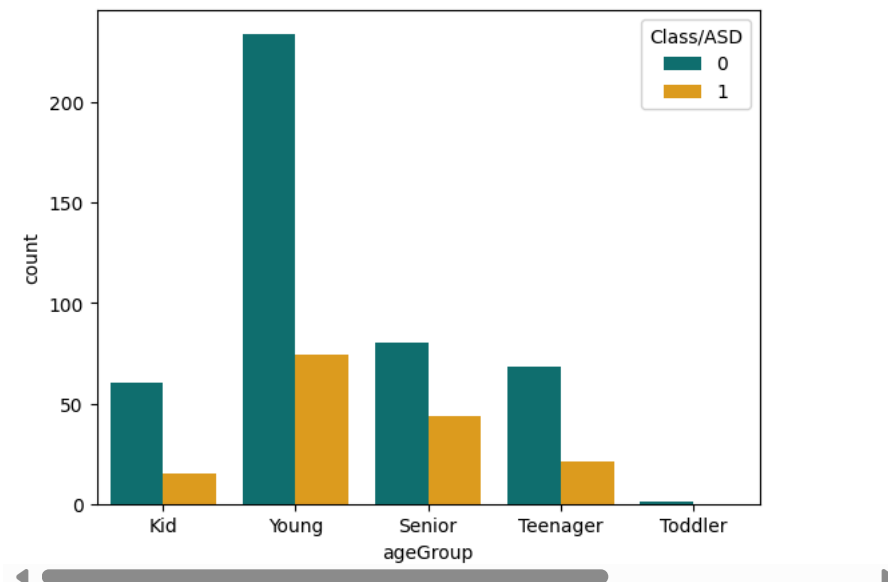
```
def convertAge(age):
    if age < 4:
        return 'Toddler'
    elif age < 12:
        return 'Kid'
    elif age < 18:
        return 'Teenager'
    elif age < 40:
        return 'Young'
    else:
        return 'Senior'

training_data['ageGroup'] = training_data['age'].apply(convertAge)

colors = ['#008080', '#FFA700', '#FF7F50']

sns.countplot(x=training_data['ageGroup'], hue=training_data['Class/ASD'], palette=colors)
plt.show()
```

```
<ipython-input-35-de3b73b08989>:3: UserWarning: The palette list has more values (3) than the number of categories (2)
sns.countplot(x=training_data['ageGroup'], hue=training_data['Class/ASD'], palette=col
```



ANALYSIS:

The above countplot shows a distribution of ASD across different age groups. People with ages between 18 and 40 that is the young people have shown more number of cases of this disorder, followed by the Senior age group, that is people having ages more than 40 years. It can also be seen that toddlers have recorded the least number of cases followed by kids and then teenagers.

```
training_data.columns
```

```
Index(['ID', 'A1_Score', 'A2_Score', 'A3_Score', 'A4_Score', 'A5_Score',
      'A6_Score', 'A7_Score', 'A8_Score', 'A9_Score', 'A10_Score', 'age',
      'gender', 'ethnicity', 'jaundice', 'austim', 'contry_of_res',
      'used_app_before', 'result', 'age_desc', 'relation', 'Class/ASD',
      'gender_encoded', 'country_encoded', 'ethnicity_encoded',
      'jaundice_encoded', 'austim_encoded', 'ageGroup'],
      dtype='object')
```

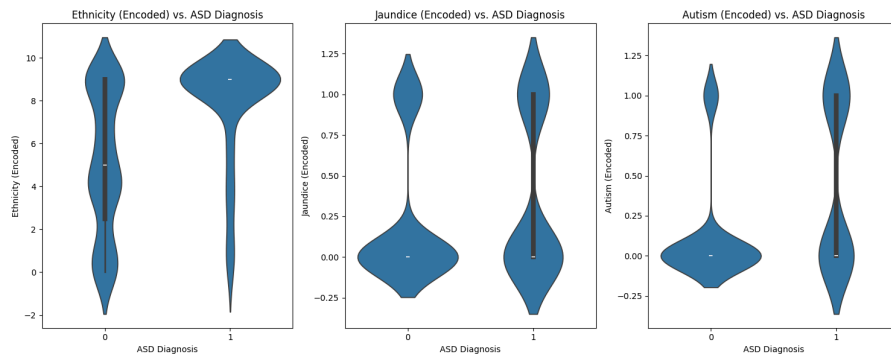
```
# Set up the figure with subplots
fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(15, 6))

# Plot violin plot for ethnicity_encoded vs. ASD diagnosis
sns.violinplot(x='Class/ASD', y='ethnicity_encoded', data=training_data, ax=axes[0])
axes[0].set_xlabel('ASD Diagnosis')
axes[0].set_ylabel('Ethnicity (Encoded)')
axes[0].set_title('Ethnicity (Encoded) vs. ASD Diagnosis')

# Plot violin plot for jaundice_encoded vs. ASD diagnosis
sns.violinplot(x='Class/ASD', y='jaundice_encoded', data=training_data, ax=axes[1])
axes[1].set_xlabel('ASD Diagnosis')
axes[1].set_ylabel('Jaundice (Encoded)')
axes[1].set_title('Jaundice (Encoded) vs. ASD Diagnosis')

# Plot violin plot for autism_encoded vs. ASD diagnosis
sns.violinplot(x='Class/ASD', y='austim_encoded', data=training_data, ax=axes[2])
axes[2].set_xlabel('ASD Diagnosis')
axes[2].set_ylabel('Autism (Encoded)')
axes[2].set_title('Autism (Encoded) vs. ASD Diagnosis')

plt.tight_layout()
plt.show()
```



✓ ANALYSIS:

The above is a violin plot which shows the correlation of columns like history of jaundice(jaundice_encoded), history of autism(austim_encoded) and ehnicity(ethnicity_encoded) with the likelihood of being detectd with ASD. The areaas where there is a fatter part shows that there is more density and similary the thinner part depicts lower density.

- **Ethnicity:** There appears to be a slightly higher density of ASD diagnoses for individuals with encoded ethnicity values of 2, 4, and 6 when compared to other values. However, the differences are not substantial, suggesting that ethnicity may not be a strong predictor of ASD.
- **Jaundice:** Individuals with encoded jaundice values of 0 and 1 seem to have a higher likelihood of being diagnosed with ASD compared to those with values of 2 and 3. This could indicate a potential association between jaundice and ASD, although further investigation is needed.
- **Autism:** The violin plot for autism shows a clear distinction between the encoded values. Individuals with an encoded autism value of 1 have a significantly higher density of ASD diagnoses compared to those with values of 0 and 2. This suggests a strong association between a history of autism and the likelihood of being diagnosed with ASD.